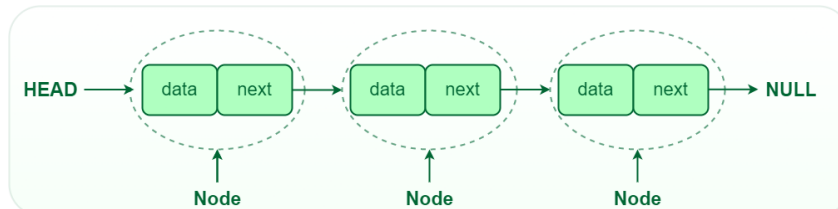


Linked list Introduction

Linked List is a linear data structure, in which elements are not stored at a contiguous location, rather they are linked using pointers. Linked List forms a series of connected nodes, where each node stores the data and the address of the next node.

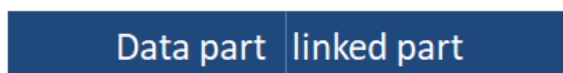


Limitation of arrays

- Array are simple to understand and elements of an array are easily accessible.
- But array have some limitations.
- Array have a fixed dimensions.
- Once the size of array is decided it can not increased or decreased during execution.
- Array element are always to stored in contiguous memory locations.
- Operations like insertion and deletion of the array are too slow.
- To overcome this limitations we use linked list

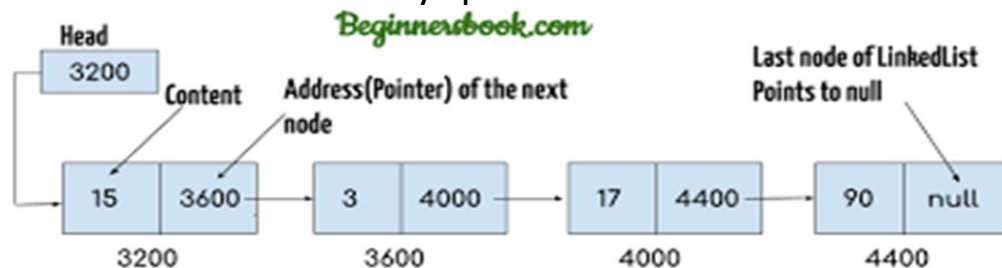
Linked List

- **Linked List** is a linear data structure
- Linked list is a collection of element called node.
- Each node contains two parts i.e data part and linked part.



- The data contains elements and link contains address of another node.

- Linked list is a data structure which can change during execution.
- Successive elements are connected by pointers last element points to null.
- It can be made just as long as required.
- It does not waste memory space.



Keep tracking of linked list:

- Must know the pointer to the first element of the list called head or start.
- Linked list contains a link element called first and each link carries a data item.
- Entry point into the linked list is called head of the list.
- Each link field is called next and each link is linked with its next link.
- Last link carries a link to null to mark the end of the list.
- Linked list provide flexibility in allowing the items to be rearranged efficiently.
- Insert an element.
- Delete an element.

Difference between Array and Linked list

ARRAY	LINKED LISTS
1. Arrays are stored in contiguous location.	1. Linked lists are not stored in contiguous location.
2. Fixed in size.	2. Dynamic in size.
3. Memory is allocated at compile time.	3. Memory is allocated at run time.
4. Uses less memory than linked lists.	4. Uses more memory because it stores both data and the address of next node.
5. Elements can be accessed easily.	5. Element accessing requires the traversal of whole linked list.
6. Insertion and deletion operation takes time.	6. Insertion and deletion operation is faster.

Representation of linked list

Struct node

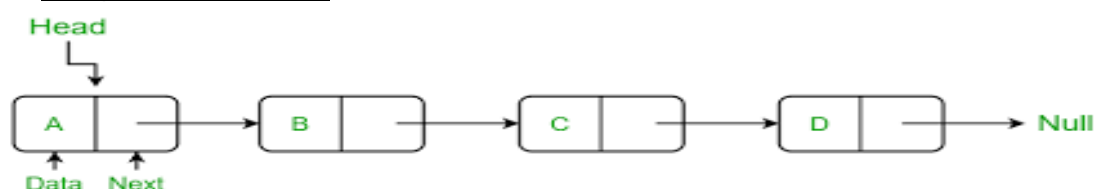
```
{
  Int data;
  Struct node*next;
};
```

Types of linked list

There are mainly three types of linked lists:

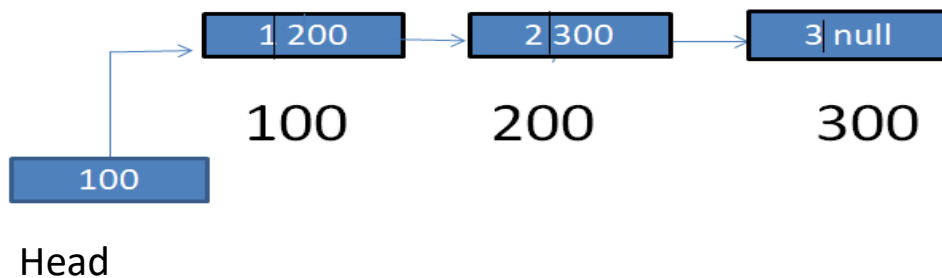
- Single-linked list
- Double linked list
- Circular linked list

1. Single-linked list:



Singly linked list is called as simple list.

- A singly linked list is one which all nodes are linked together in some sequential manner.
- The singly linked list a data structure that contains two parts.
- One is data part and another one is address part.
- Which contains the address of the next or the success or node.
- The address part in a node is also know as a pointer.
- The list can traverse in forward direction.



We can observe in the above figure that there are three different nodes having address 100,200,&300, respectively. i.e 200, is the second node contains the address of the last node i.e 300 and the third node contains the null value in its address part as it does not point to any node.

The pointer that holds the address of the initial node is known as a head pointer.

Representation of node in a singly linked list

Struct node

```

{
  Int data;
  Struct node*next;
}
  
```

In the above representation, we have defined a containing two members , the first one is data of integer type, & the other one is the pointer of the node type.

Operation on singly linked list

- Node creation
- Insertion
- Deletion and traversing

1.Node creation

Struct node

```

{
    int data ;
    struct node * next ;
    };
    struct node * head , * ptr ;
    ptr = ( struct node ) malloc ( size of ( struct node ) ) ;

```

2.Insertion

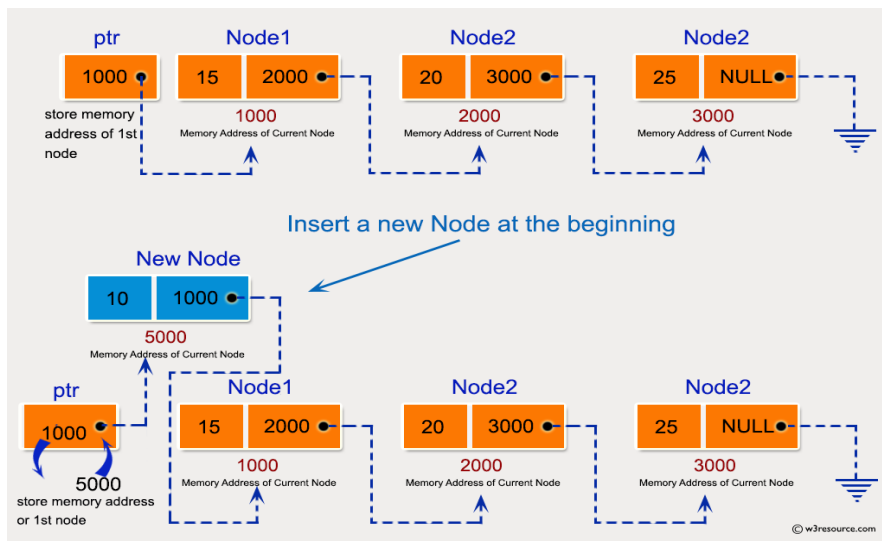
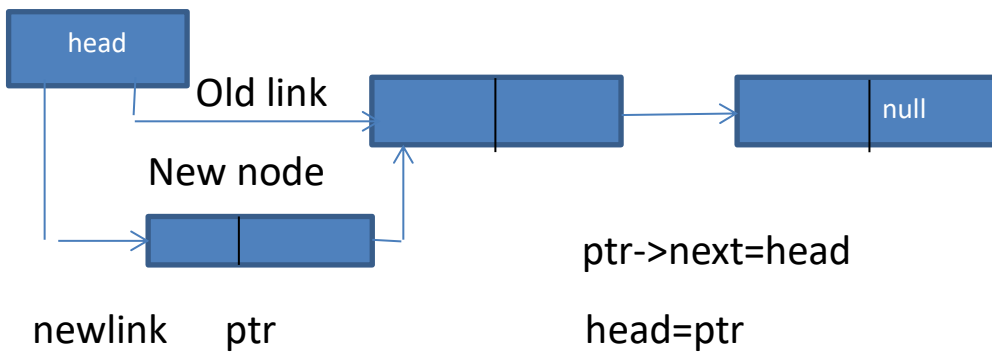
The insertion into a singly linked list can be performed at different positions .Based on the position of the new node being inserted is categorized into the following categories.

1. Insertion at beginnings
2. Insertion at end of the list
3. Insertion after specified node

Insertion at beginning

- Inserting a new element into a singly linked list at beginning is quite simple. we just need to make a few adjustment is the nodes link.
- There are the following steps which need to be followed in order to insert a new node in the list at beginning.
- Allocate the space for the new node and store data into the data part of the node.
- This will be done by the following statement,
- `ptr=(struct node*) malloc(size of(struct node*))`;
- `ptr->data=item`. Make the link part of the new node pointing to the existing first node of the list.
- This will be done by using the following statements
- `ptr->next=head`;
- At the last, we need to make the new node as by using the following statements.
- `Head=ptr`;
- Allocate the space for the new node and store data into the data part of the node.
- This will be done by the following statement,
- `ptr=(struct node*)malloc(size of(struct node*))`;

- ptr-data=item.



Ptr->next=head

Head=ptr

Algorithm

step-1: IF PTR=NULL

Write OVERFLOW

Go to step 7

[END OF IF]

Step-2: SET NEW NODE = PTR

Step-3: SET PTR =PTR NEXT

Step-4: SET NEW CODE DATA =VAL

Step-5: SET NEW NODE NEXT =HEAD

Step-6: SET HEAD =NEW CODE

Step-7: EXIT

Insertion at the end of the list

1. In order to insert a node at the last, there are two following scenarios which need to be mentioned.
2. The node is being added to an empty list .
3. The node is being added to the end of the linked list in the first case.
4. The condition (head==NULL) gets satisfied . Hence , we just need to allocate the space for the node by using malloc statement in C.
5. Ptr = (struct node) malloc (size of (struct node *));
6. Data and the link part of the node are set up by using the following statements.
7. Ptr ->data = item;
ptr ->next= NULL;
8. Since, ptr is the only node that will be inserted in the list hence, we need to make this node pointed by the head pointer of the list . This will be done by using the following statements.
9. Head= ptr

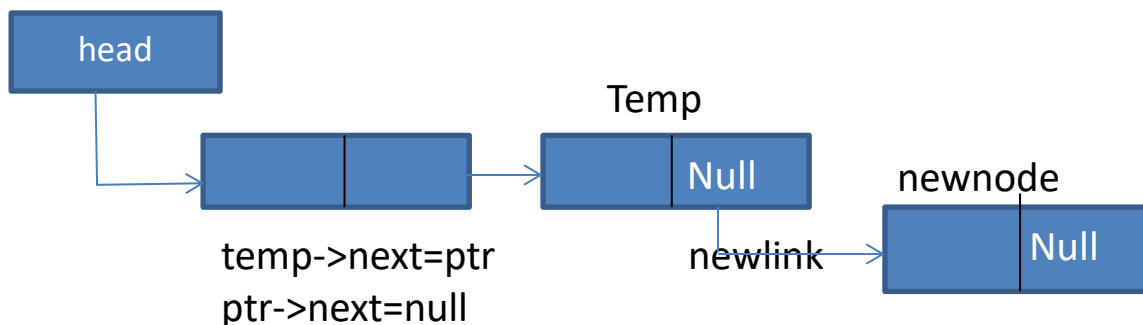
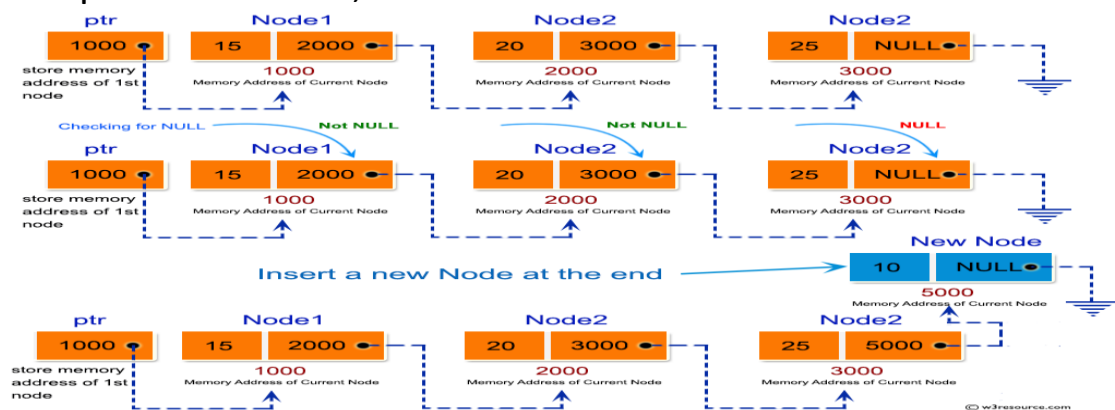
second scenario

- 1.The condition Head=NULL would fail, since head is not null.
- 2.Now, we need to declare a temporary pointer temp in order to traverse through the list. Temp is made to point the first node of the list.
- 3.Temp=head
- 4.Then, traverse through entire linked list using the statements:
- 5.At the end of the loop , the temp will be pointing to the last node of the list .
- 6.Now , allocate the space for the new node and assign the item to its data part.
- 7.Ptr= (struct node*) malloc (sizeof (struct node*));
- 8.Ptr-> data =item;
- 9.Since ,the new node is going to be the last node of the list hence, the next part of this node needs to be pointing to the null.
- 10.We need to make the next part of the temp node (which is currently the last node of the list) point to the new node (ptr) .

```

temp=head;
while ( temp next != NULL )
{
temp=temp->next;
}
temp->next=ptr;
ptr->next=NULL;

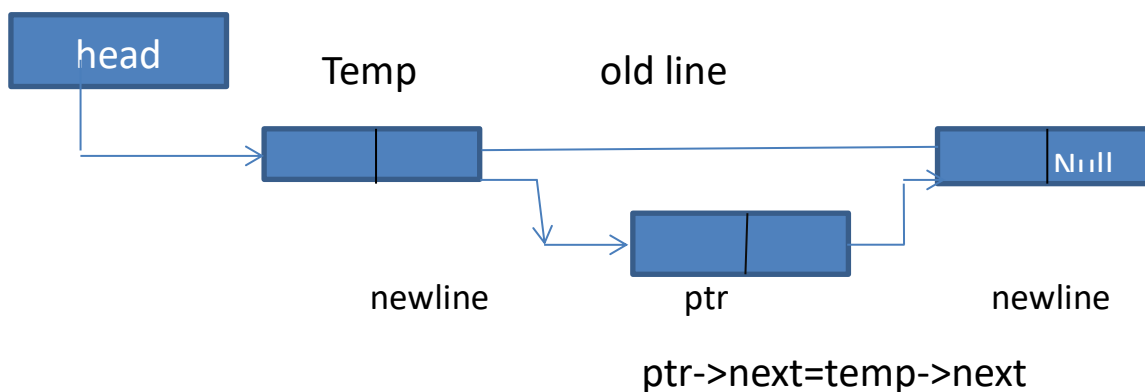
```



Inserting after specified node

1. In order to insert an element after the specified number of nodes into the linked list, we need to skip the desired number of elements in the list to move the pointer at the position after which the node will be inserted.
2. Allocate the space for the new node & add the item to the data part of it. This will be done by using the following statements.
3. `Ptr=(struct node*)malloc(sizeof(struct node));`
4. `Ptr->data=item;`
5. Now, we just need to make a few more link adjustments and our node will be inserted at the specified position. Since, at the end of the loop, the node after which the new node will be inserted.

6. Therefore, the next part of the new node ptr must contain the address of the next part of the temp (since , ptr will be in between temp & the next of the temp).
7. This will be done by using the following statements.
Ptr->next=temp->next
8. Now, we just need to make the part of the temp, point to the new node ptr.
9. This will insert the new node ptr, at the specified position.
Temp->next=ptr;



temp->next=ptr;

Deletion and Traversing

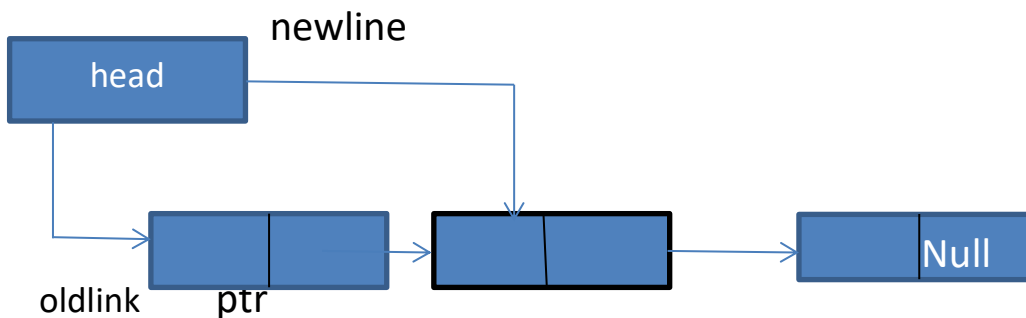
1. The deletion of a node from a singly linked list can be performed at different positions.
2. Based on the position of the node being deleted, the operation is categorized into the following categories.
 - Deletion at beginning
 - Deletion at the end of the list
 - Deletion after specified node

Deletion in singly linked list at beginning

1. Deleting a node from the beginning of the list is the simplest operation of all.
2. It just need a few adjustment in the node pointers.
3. Since the first node of the list is to be deleted , therefore , we just need to make the head, point to the next of the head.
4. This will be done by using the following statements.
5. Ptr = head;
6. Head = ptr->next;

7. Now ,free the pointer which was pointing to the head of the list .this will be done by using the following statement.

8. Free (Ptr)



```
ptr=head
```

```
head=ptr->next
```

```
free(ptr)
```

Deleting node from the beginning

Algorithm:

Step1: IF HEAD=NULL

GOTO STEP 5

[END OF IF]

Step2: SET PTR=HEAD

Step3: SET HEAD=PTR->NEXT

Step4: FREE PTR

Step5: EXIT

Traversing is singly linked list

Traversing is the most common operation that is performed in almost every scenario of singly linked list. Traversing means visiting each node of the list once in order to perform some operation on that.

This will be done by using the following statements.

```
Ptr=head;
```

```
While(ptr->next!=NULL)
```

```
{
```

```
Ptr=ptr->next;
```

```
}
```

Algorithm

Step1: SET PTR=HEAD

Step2: IF PTR=NULL

WRITE "EMPTY LIST"

GOTO STEP 6

END OF IF

Step3:REPEAT STEP4 AND 5 UNTILL PTR->NEXT!=NULL

Step4:PRINT PTR->DATA

Step5:PTR=PTR->NEXT

[END OF LOOP]

Step6:EXIT

Searching

Searching is performed in order to find the location of a particular element in the list.

Searching any element in the list needs traversing through the list and make the comparison of every element of the list with the specified element.

If the element is matched with any of the list element is matched The location of the element is returned from the function.

Algorithm

Step1:SET PTR=HEAD

Step2:SET L=0

Step3:IF PTR=NULL

WRITE "EMPTY LIST"

GOTO STEP 8

END OF IF

Step4:REPEAT STEP 5 TO STEP 7 UNTILL PTR->NEXT!=NULL

Step5:IF PTR->DATA=ITEM

WRITE L+1

END OF IF

Step6:L=L+1

Step7:PTR=PTR->NEXT

(END OF LOOP)

Step 8:EXIT

3. Doubly Linked List

Doubly linked list is a complex type of linked list in which contains a pointer to the previous as well as the next node in sequence.

Therefore, in doubly linked list, node consists of three parts: node data, pointer to the next node in sequence(next pointer), pointer to the previous node(previous pointer).

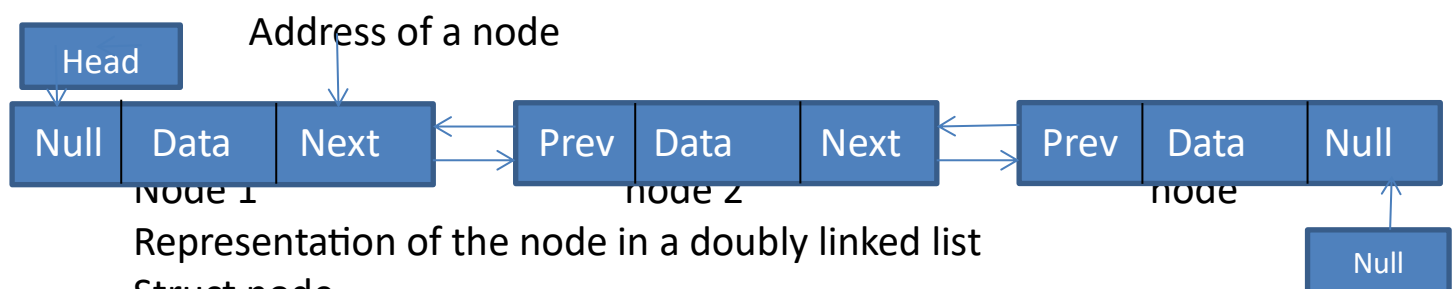


A singly linked list has some disadvantages that it can traverse in one direction.

Pointers exist between adjacent nodes in both directions.

The list can be traversed either forward or backward.

Usually two pointers are maintained to keep track of the list, head and tail.



Representation of the node in a doubly linked list

Struct node

```
{
Struct node *prev;
Int data;
Struct node *next;
}
```

The prev part of the first node and the next part of the last node will always contain null indicating end in each direction.

Operation on doubly linked list

1. Node Creation
2. Insertion
3. Deletion and Traversing

1. Node Creation

Struct node

```
{
Struct node *prev;
Int data;
Struct node*next;
};
Strcut node *head;
```

2.Insertion

The insertion into a singly linked list can be performed at different positions.

Based on the position of the new node being inserted, the insertion is categorized into the following categories.

- Insertion at beginning
- Insertion at the end of the list
- Insertion after specified node

Insertion at beginning

As in doubly linked list , each node of the list contain double pointers. Therefore we have to maintain more numbers of pointers in doubly linked list as compare to singly linked list.

There are two scenario of inserting any element into doubly linked list. Either the list is empty or it contains at least one element.

Perform the following steps to insert a node in doubly linked list beginning.

Allocate the space for the new node in the memory. This will be done by using the following statement.

```
Ptr=(struct node*) malloc (sizeof (struct node));
```

Check whether the list is empty or not, the list is empty if the condition `head==null` holds.

In that case, the node will be inserted at the only node of the list and therefore the prev and the next pointer of the node will point to null and the head pointer will point to this node.

```
Ptr->next=NULL;
```

```
Ptr->prev=NULL;
```

```
Ptr->data=ITEM;
```

```
Head=ptr;
```

In the second scenario , the condition `head=null` become false and the node will be inserted in beginning . the next pointer of the node will point to the existing head pointer of the node, the prev pointer of the existing head will point to the new node being inserted.

This will be done by using the following statements.

```
Ptr->next=head;
```

Head->prev= ptr;

Since, the node being inserted is the first node of the list and therefore it must contain null in its prev pointer. Hence assign null to its previous part and make the head point to this node.

Ptr->prev=NULL

Head=ptr

Algorithm

Step1:IF PTR=NULL

WRITE OVERFLOW

GO TO STEP 9

END OF IF

Step2:NEW NODE=PTR

Step3:PTR=PTR->NEXT

Step4:SET NEW NODE->DATA=VAL

Step5:SET NEW NODE->PREV=NULL

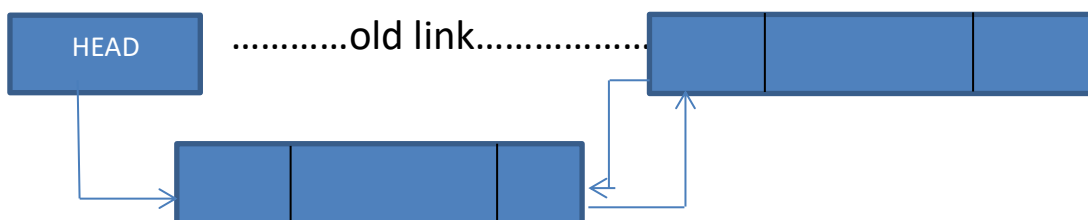
Step6 :SET NEW NODE->PREV=NULL

Step7:SET HEAD->PREV=NEWNODE

Step8:SET HEAD=NEW NODE

Step9:EXIT

Insertion into doubly linked list at beginning



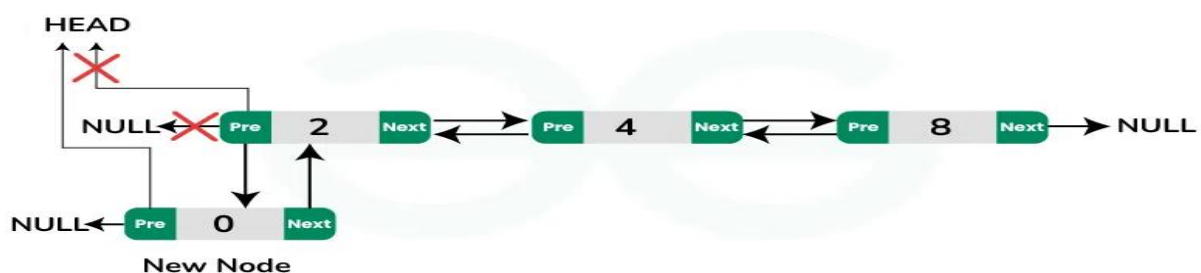
Newlink New node

Ptr->prev=NULL

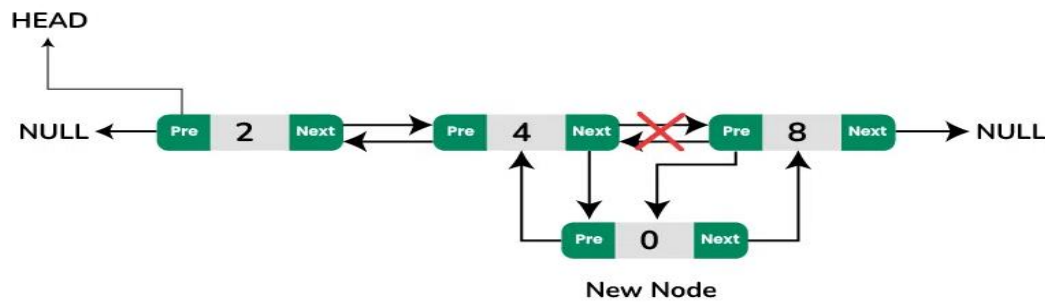
Ptr->next=head

Head->prev=ptr

Head=ptr



Insertion in between two nodes in Doubly Linked List:



Insertion at the end of Doubly Linked List:

Deletion at Beginning

Deletion in doubly linked list at the beginning is the simplest operation we just need to copy the head pointer to pointer ptr & shift the head pointer to its next.

Ptr=head;

Head=head->next;

Now make the prev of this new head node point to NULL. This will be done by using the following statements.

Head->prev=NULL

Now free the pointer ptr by using the free function free (ptr)

Algorithm

Step1:IF HEAD=NULL

WRITE UNDERFLOW GOTO STEP 6

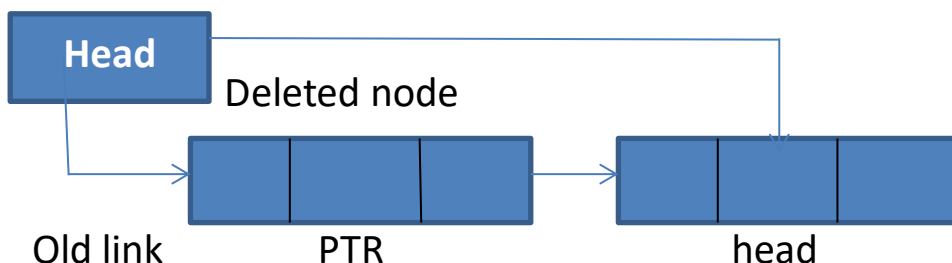
Step2:SET PTR=HEAD

Step3: SET HEAD=HEAD->NEXT

Step4:SET HEAD->PREV=NULL

Step5:FREE PTR

Step6:EXIT



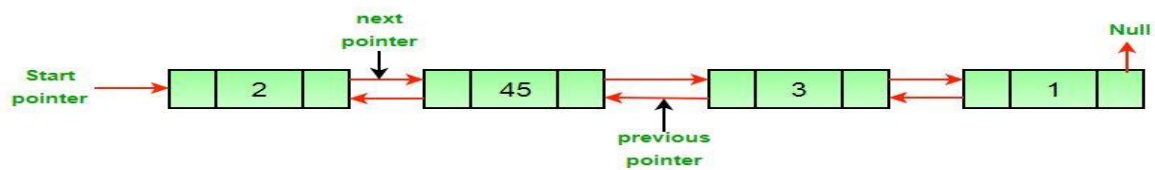
Ptr=head

Head=head->next

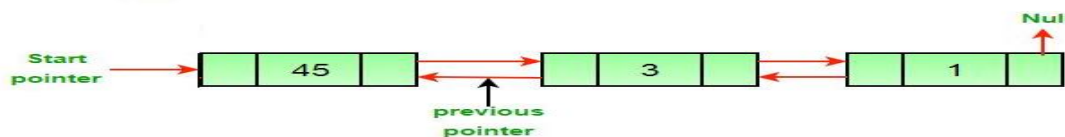
Head->prev=NULL

Free(ptr)

Delete a node in a Doubly Linked List



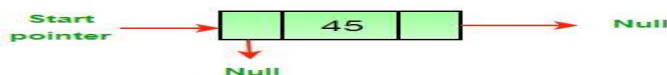
After the deletion of the head node



After deletion of the middle node



After delete the last node



Searching for a specific node in doubly linked list

We just need traverse the list in order to search for a specific element in the list. Perform following operations in order to search a specific operation.

Copy head pointer into a temporary pointer variable ptr.

Ptr =head

Declare local variable i and assign it to 0.

i=0

Traverse the list until the pointer ptr become null. Keep shifting pointer to its next & increasing i by+1.

Compare each element of the list with the item which is to be searched.

If the item matched with any node value then the location of that value1 will be returned from the function else null is returned.

Algorithm

Step1: IF HEAD =NULL

 WRITE "UNDERFLOW"

 GO TO STEP 8

 (END OF IF)

Step2: SET PTR=HEAD

Step3: SET i=0

Step4: REPEAT STEP 5 TO 7 WHILE PTR!=NULL

Step5: IF PTR->DATA=ITEM

 RETURN i

 (END OF IF)

Step6: i=i+1

Step7: PTR=PTR->NEXT

Step8: EXIT

Traversing in doubly linked list

Traversing is the most common operation in case of each data structure. for this purpose , copy the head pointer in any of the temporary pointer ptr.

Ptr=head

Then , traverse through the list by using while loop. Keep shifting value of pointer variable ptr until we find the last node. The last node contain null in its next part.

While(ptr!=null)

{

 Printf("%d/n", ptr->data);

 Ptr=ptr->next;

}

Although , traversing means visiting each node of the list once to perform some specific operation . here, we are printing the data associated with each node of the list.

Algorithm

Step1: IF HEAD=NULL

 WRITE "UNDERFLOW"

GO TO STEP 6

(END OF IF)

Step2: SET PTR=HEAD

Step3: REPEAT STEP 4 AND 5 WHILE PTR!=NULL

Step4: WRITE PTR->DATA

Step5: PTR->PTR->NEXT

Step6: EXIT

3.Circular linked list

A circular linked list is a variation of a singly linked list . the singly linked list and a circular linked list is that the last node does not point to any node in a singly linked list, so its link part contains a null value. On the other hand, the circular linked list is a list in which the last node concepts to the first node, so the link part of the last node holds the first nodes address.

The circular linked list has no starting & ending node. We can traverse in any direction, i.e either backward or forward.

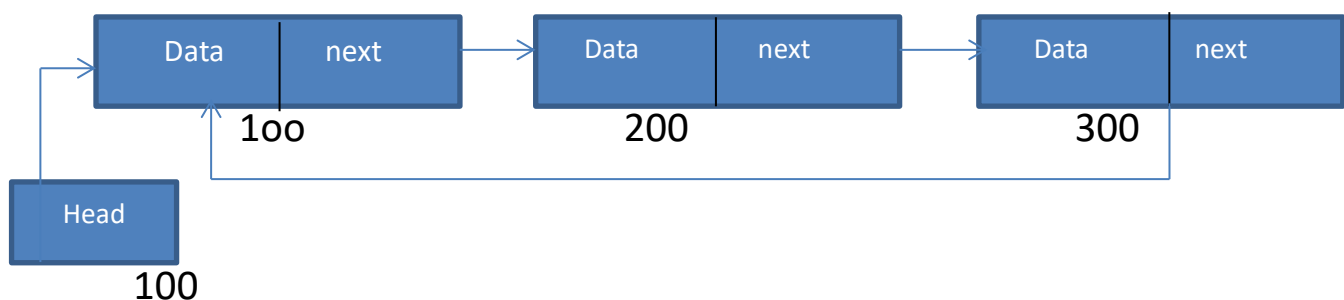
A circular linked list is a sequence of elements in which each node has a link to the next node, and the last node is having a link to the circular linked list, as shown below: either backward or forward.

Struct node

{

Int data;

Struct node *next



TREE

A tree is a nonlinear data structure.

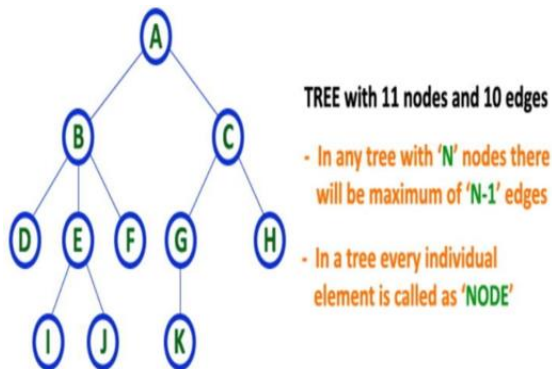
Tree is also a collection of vertices and edge. However in tree data structure, There can only be one edge between two vertices.

A tree is a collection of nodes connected by directed(or undirected) edges.

The topmost node of the tree is called the root, and the nodes below it are called the child nodes.

Popular tree based data structure

1. Binary tree
2. Binary search tree



Terminology

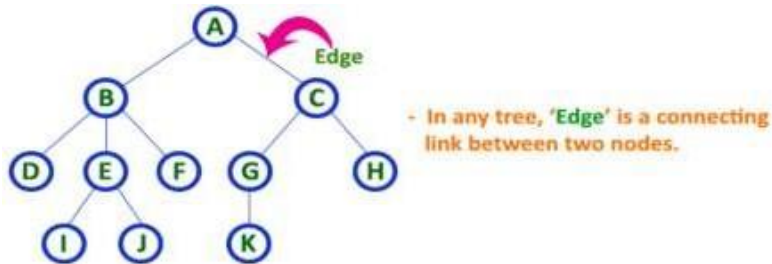
Root

In a tree data structure, the first node is called as **RootNode**. Every tree must have a root node.



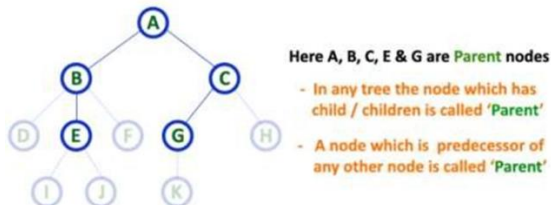
Edge

In a tree data structure connecting link between two nodes called edge.



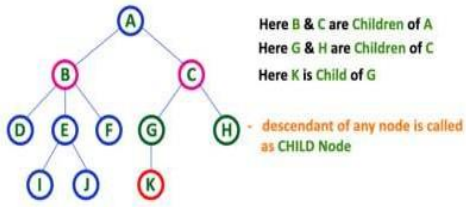
Parent node

In a tree data structure node which has a branch from it to any other node is called a PARENT NODE.



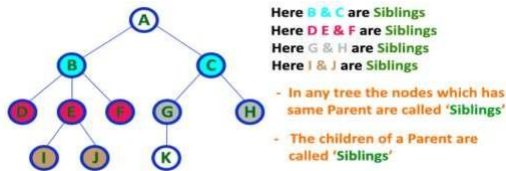
Child

In a tree data structure, node which has a link from its parent node is called as child node.



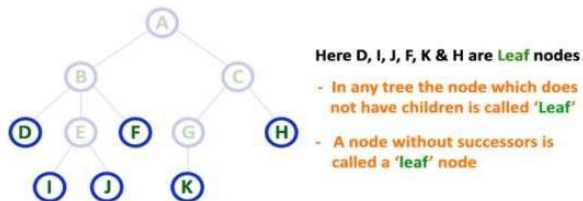
Siblings

In a tree data structure, nodes which belong to same Parent are called as **SIBLINGS**.



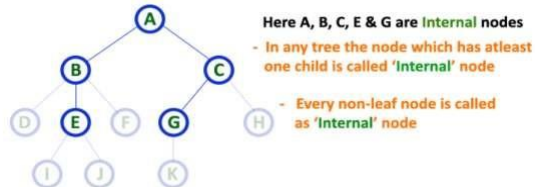
Leaf

In a tree data structure, the node which does not have a child is called as **LEAF Node**.



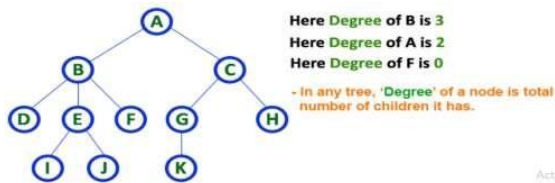
Internal Nodes

In a tree data structure, the node which has atleast one child is called as **INTERNAL Node**.



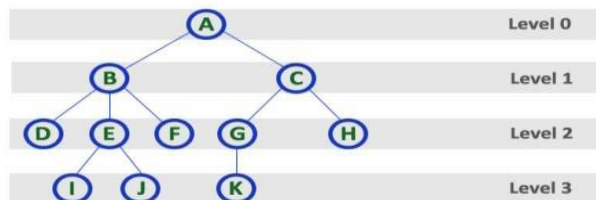
Degree

In a tree data structure, the total number of children of a node is called as **DEGREE** of that Node.



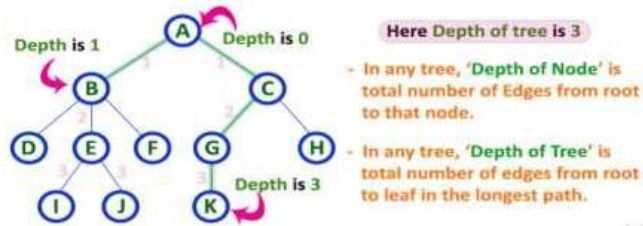
Level

In simple words, in a tree each step from top to bottom is called as a Level



Depth

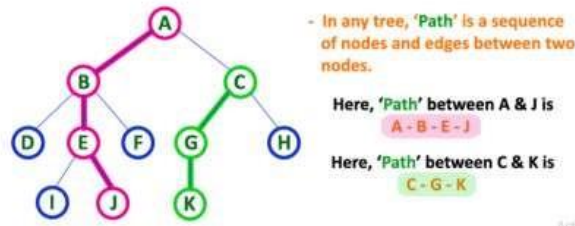
In a tree data structure the total number of edges from root node to a leaf node in the longest path is said to be **Depth of the tree**.



Path

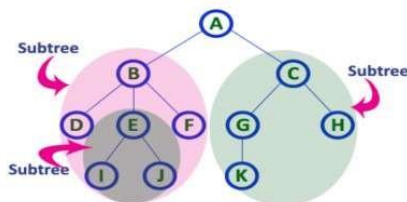
In a tree data structure, the sequence of Nodes and Edges from one node to another node is called

as **PATH** between that two Nodes.



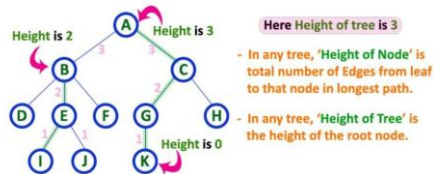
Sub Tree

In a tree data structure, each child from a node forms a sub tree recursively. Every child node will form a sub tree on its parent node.



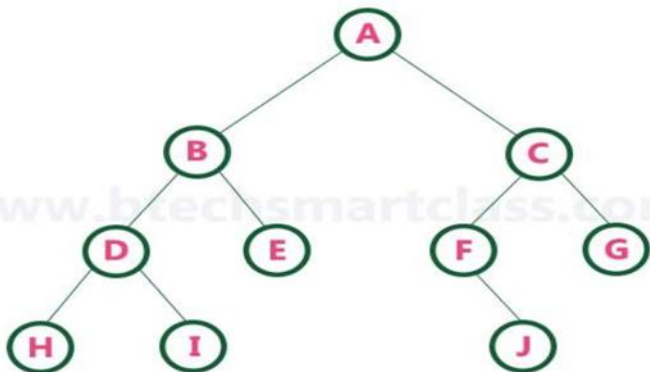
Height

In a tree data structure, the total number of edges from leaf node to a particular node in the longest path is called as **HEIGHT** of that Node.



Binary Tree

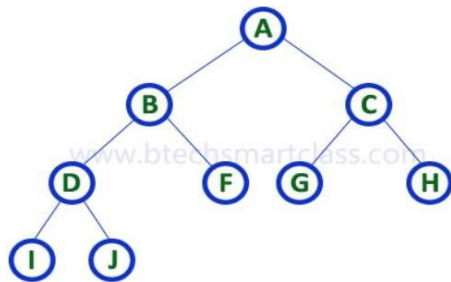
- A binary tree is a special type of tree data structure in which every node can have a **maximum of 2 children**.
- One is known as a left child and the other is known as right child.
- In a binary tree, every node can have either 0 children or 1 child or 2 children but not more than 2 children.



- Here, 'A' is the root node of the binary tree.
- B is the left child of 'A' and 'C' is the right child of 'A'.
- i. e 'A' is a father of B and C.
- The node B and C are called siblings.
- Node H, I, E, J, G, are leaf node.

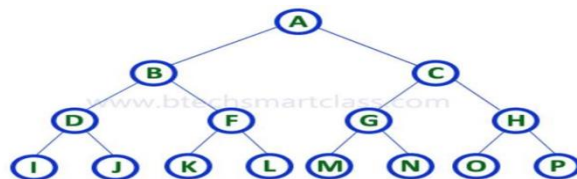
1. Strictly Binary Tree

- A binary tree in which every node has either two or zero number of children is called Strictly
- Strictly binary tree, every node should have exactly two children or none. That means every internal node must have exactly two children.
- Strictly binary tree is also called as **Full Binary Tree** or **Proper Binary Tree**.



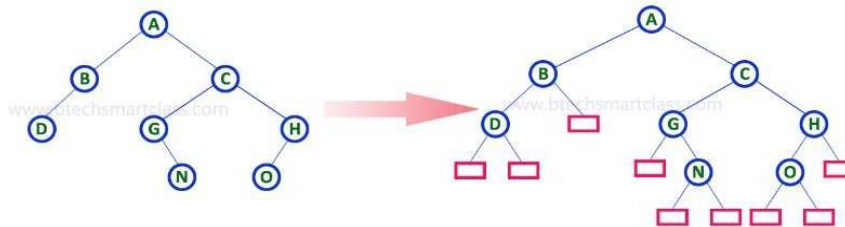
2. Complete Binary Tree

- binary tree in which every internal node has exactly two children and all leaf nodes are at same level is called Complete Binary Tree.
- For example at level 2 there must be $2^2 = 4$ nodes and at level 3 there must be $2^3 = 8$ nodes.
- Complete binary tree is also called as Perfect Binary Tree



3. Extended Binary Tree

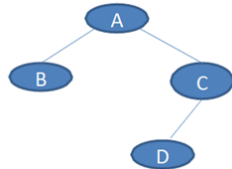
The full binary tree obtained by adding dummy nodes to a binary tree is called as Extended Binary tree.



- In above figure, a normal binary tree is converted into full binary tree by adding dummy nodes (In pink colour).

4. Almost Complete Binary Tree

An almost complete binary tree is a binary tree where for a right child , there is a left child, but for a left child there may not be a right child.



TREE NODE

A node is a structure that contains a key or value and pointers in its child node in the tree data structure.

In the tree data structure, you can define the tree node as follows,

```
Struct node
{
    int data;
    Struct node*left child;
    Struct node*right child;
};
```

Operation on Binary Tree

- **Create:** Create an empty tree.
- **Insert:** Insert a node into a tree.
- **Search:** Searches for a node in a tree.
- **Delete:** Deletes a node from the tree.
- **In-order:** In-order traversal of the tree.
- **Pre-order:** In-order traversal of the tree.
- **Post-order:** In-order traversal of the tree.

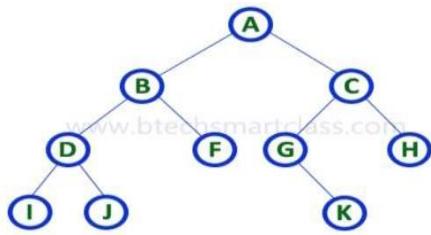
Binary Tree Traversal

- Displaying (or) visiting order of nodes in a binary tree is called as Binary Tree Traversal.
- Traversal is a process to visit all the nodes of a tree and may print their values too.
- All nodes are connected via edges (link) we always start from root node.

There are three types of binary tree traversals.

- **In - Order Traversal**
- **Pre - Order Traversal**
- **Post - Order Traversal**

Consider the following binary tree..



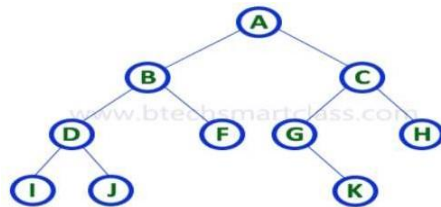
1. **In - Order Traversal (left Child - root – right Child)** In In-Order traversal, the root node is visited between the left child and right child. In this traversal, the left child node is visited first, then the root node is visited and later we go for visiting the right child node.

In-Order Traversal for above example of binary tree is I - D - J - B - F - A - G - K - C - H

2. **Pre - Order Traversal (root – left Child – right Child)**

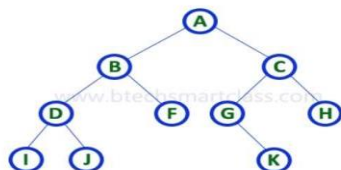
In Pre-Order traversal, the root node is visited before the left child and right child nodes. In this traversal, the root node is visited first, then its left child and later its right child.

Pre-Order Traversal for above example binary tree is A - B - D - I - J - F - C - G - K - H



3. **Post - Order Traversal (left Child – right Child - root)**

In Post-Order traversal, the root node is visited after left child and right child. In this traversal, left child node is visited first, then its right child and then its root node. This is recursively performed until the right most node is visited.



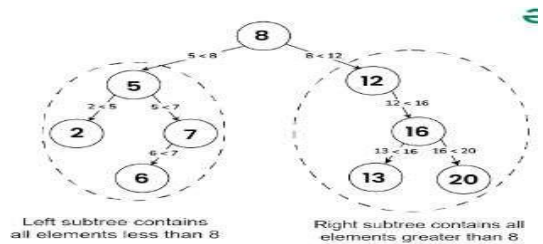
Post-Order Traversal for above example binary tree is I - J - D - F - B - K - G - H - C - A

Binary Search Tree

A binary search tree is a binary tree that is either empty or in which every node contains a key value.

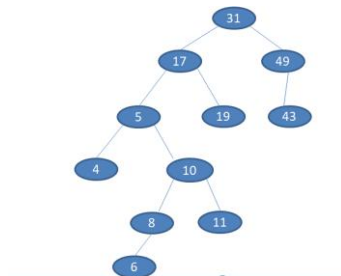
Binary search tree follows properties :

- Binary tree and its **left** child contains values less than the parent node .
- **right** child contains values greater than the parent node.

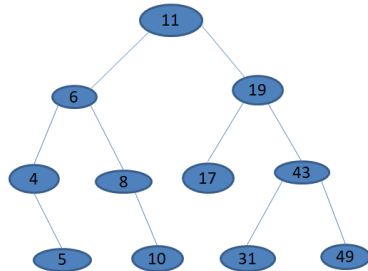


- A binary search tree is basically a binary tree, and therefore it can be traversed in in-order , pre-order, post-order.
- If we traverse a binary search tree in in-order and print the identifiers contained in the nodes of the tree, we get a sorted list of identifiers in ascending order.
- Draw a binary search tree by inserting following numbers from left to right.

11,6,8,19,4,10,5,17,43,49,31.



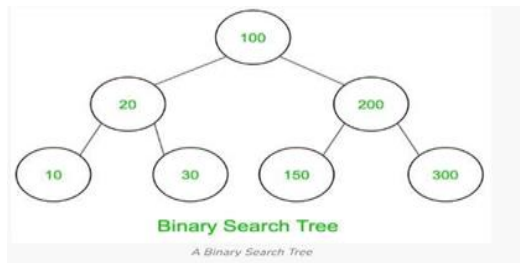
- Draw a binary search tree by inserting following numbers from right to left.
11,6,8,19,4,10,5,17,43,49,31.



Binary Search Tree (BST) Traversals

Given a [Binary Search Tree](#), The task is to print the elements in in-order, pre-order, and post-order traversal of the Binary Search Tree.

- In-order traversal
- Pre-order traversal
- Post-order traversal
-



- ***In-order Traversal: 10 20 30 100 150 200 300***
- ***Pre-order Traversal: 100 20 10 30 200 150 300***
- ***Post-order Traversal: 10 30 20 150 300 200 100***

